

EC2- Configuring Apache/Nginx Web Server in Linux Instance

Project Overview

Project Name: Web Server Deployment on AWS EC2 **Description:** This project involves setting up an Apache or Nginx web server on an AWS EC2 Linux instance to host web applications.

Owner: [Jeevan Pariyar] **Date:** [15-03-2025]

AWS Services Used

The screenshot displays the AWS Management Console interface for the EC2 service. The left-hand navigation pane shows the 'Instances' section selected. The main content area shows the details for a single EC2 instance, 'i-06adf6e88f21c133e', which is in the 'Running' state. The instance is a 't2.micro' type, located in the 'ap-south-1b' availability zone. The public IP address is '13.201.184.253'. The instance is running on the 'Ubuntu Server 20.04 LTS' AMI. The console also shows the instance's network configuration, including the VPC ID 'vpc-04e08b28dcb198f2' and the subnet ID 'subnet-07023843c5995e439'. The instance is associated with the 'Default' security group. The console also shows the instance's monitoring data, including CPU usage and network I/O. The bottom of the console shows the 'Network interfaces' section, which lists the instance's network interfaces and their associated VPC and subnet.

Instances (1/1) info

Find Instance by attribute or tag (case-sensitive) Running

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS
Practical 2 :- EC2-Configuring Apache/...	i-06adf6e88f21c133e	Running	t2.micro	Initializing	View alarms +	ap-south-1b	ec2-13-201-184-

i-06adf6e88f21c133e (Practical 2 :- EC2-Configuring Apache/Nginx web server in Linux instance)

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

Instance summary

Instance ID: i-06adf6e88f21c133e

Public IPv4 address: 13.201.184.253 | open address

Private IPv4 addresses: 172.31.0.224

Public IPv4 DNS: ec2-13-201-184-253.ap-south-1.compute.amazonaws.com | open address

Instance state: Running

Private IP DNS name (IPv4 only): ip-172-31-0-224.ap-south-1.compute.internal

Instance type: t2.micro

VPC ID: vpc-04e08b28dcb198f2

Auto-assigned IP address: 13.201.184.253 [Public IP]

Host name type: IP name: ip-172-31-0-224.ap-south-1.compute.internal

Answer private resource DNS name: IPv4 (A)

Elastic IP addresses: -

AWS Compute Optimizer finding: Opt-in to AWS Compute Optimizer for recommendations.

Networking details

Public IPv4 address: 13.201.184.253 | open address

Public IPv4 DNS: ec2-13-201-184-253.ap-south-1.compute.amazonaws.com | open address

Subnet ID: subnet-07023843c5995e439

Availability zone: ap-south-1b

Use RBN as guest OS host name: Disabled

Private IPv4 addresses: 172.31.0.224

Private IP DNS name (IPv4 only): ip-172-31-0-224.ap-south-1.compute.internal

IPv6 addresses: -

Carrier IP addresses (ephemeral): -

Answer RBN host name IPv4: Enabled

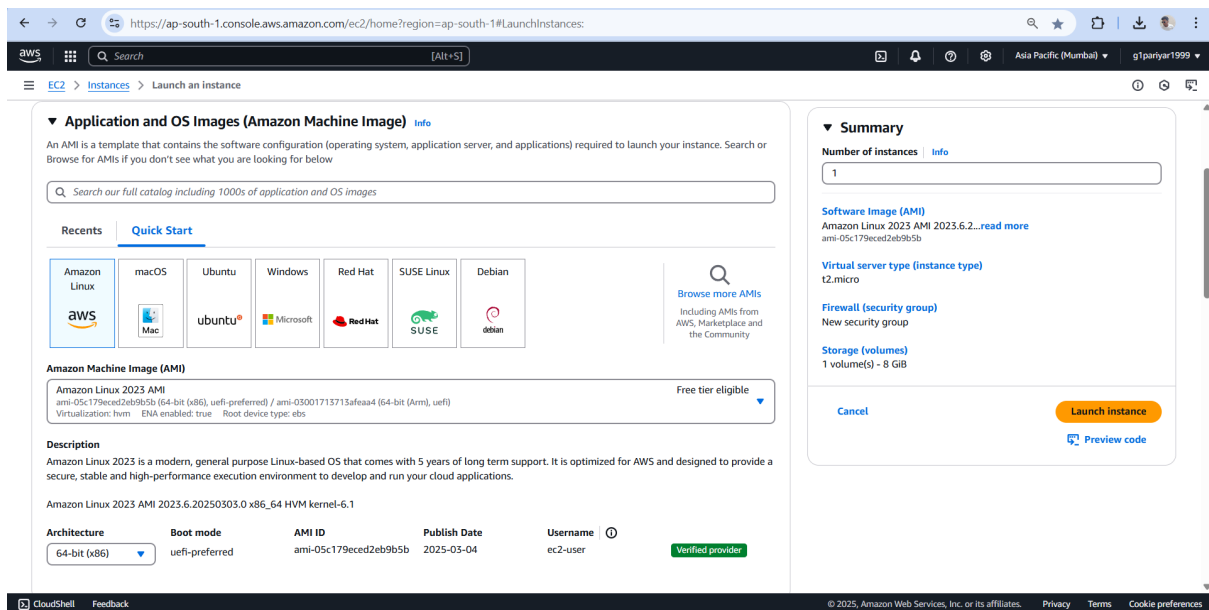
VPC ID: vpc-04e08b28dcb198f2

Secondary private IPv4 addresses: -

Outpost ID: -

Network Interfaces (1)

- **Compute: Amazon EC2 (Linux Instance)**



- **Networking: VPC, Security Groups, Elastic IP**
- **Storage: Amazon EBS**
- **Monitoring: Amazon CloudWatch**

Deployment Process

Step 1: Launch an EC2 Linux Instance

1. **Navigate to the AWS EC2 console.**
2. **Click on "Launch Instance."**
3. **Choose an Amazon Machine Image (AMI) (e.g., Amazon Linux, Ubuntu, CentOS).**
4. **Select an appropriate instance type (e.g., t2.micro for free tier eligibility).**
5. **Configure instance details:**
 - **Ensure public IP is enabled.**
 - **Assign to a public subnet in the VPC.**
6. **Add storage (default or custom EBS volume).**
7. **Configure security group:**
 - **Allow HTTP (80) and HTTPS (443) from anywhere (0.0.0.0/0) OR MY IP.**
 - **Allow SSH (22) access from trusted IPs.**
8. **Create or select an existing key pair.**
9. **Review and launch the instance.**

Step 2: Connect to the Instance

1. Use SSH to connect to the instance:

```
ssh -i your-key.pem ec2-user@your-instance-ip
```

2. Update the package repository:

3. `sudo yum update -y` # Amazon Linux

4. `sudo apt update -y` # Ubuntu/Debian

```
sudo dnf update -y # RHEL 8+
```

Step 3: Install and Configure Apache or Nginx

Install Apache (HTTPD)

1. Run the following command to install Apache:

2. `sudo yum install httpd -y` # Amazon Linux, CentOS

```
sudo apt install apache2 -y # Ubuntu/Debian
```

3. Start and enable Apache:

4. `sudo systemctl start httpd`

```
sudo systemctl enable httpd
```

5. Adjust firewall rules (if needed):

6. `sudo firewall-cmd --add-service=http --permanent`

```
sudo firewall-cmd --reload
```

7. Verify Apache is running:

```
systemctl status httpd
```

8. Place your web files in `/var/www/html/`.

Install Nginx

1. Run the following command to install Nginx:

2. `sudo yum install nginx -y` # Amazon Linux, CentOS

```
sudo apt install nginx -y # Ubuntu/Debian
```

3. Start and enable Nginx:

4. `sudo systemctl start nginx`

```
sudo systemctl enable nginx
```

5. Adjust firewall rules (if needed):

6. `sudo firewall-cmd --add-service=http --permanent`

```
sudo firewall-cmd --reload
```

7. Verify Nginx is running:

systemctl status nginx

8. Place your web files in /usr/share/nginx/html/.

```
Process: 26432 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
Process: 26442 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
Main PID: 26433 (nginx)
Tasks: 3 (limit: 1111)
Memory: 2.4M
CGroup: /system.slice/nginx.service
└─26435 "nginx: master process /usr/sbin/nginx"
   └─26450 "nginx: worker process"

Mar 15 09:10:22 ip-172-31-0-224.ap-south-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Mar 15 09:10:22 ip-172-31-0-224.ap-south-1.compute.internal nginx[26433]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Mar 15 09:10:22 ip-172-31-0-224.ap-south-1.compute.internal nginx[26433]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Mar 15 09:10:22 ip-172-31-0-224.ap-south-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
(eo2-user@ip-172-31-0-224 ~)$ sudo systemctl enable nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service - /usr/lib/systemd/system/nginx.service.
(eo2-user@ip-172-31-0-224 ~)$ cd /usr/share/nginx/html/
(eo2-user@ip-172-31-0-224 html)$ ls
404.html  50x.html  icons  index.html  nginx-logo.png  poweredby.png
(eo2-user@ip-172-31-0-224 html)$ rm -rf index.html
rm: cannot remove 'index.html': Permission denied
(eo2-user@ip-172-31-0-224 html)$ sudo rm -rf index.html
(eo2-user@ip-172-31-0-224 html)$ ls
404.html  50x.html  icons  nginx-logo.png  poweredby.png
(eo2-user@ip-172-31-0-224 html)$ cat index.html
(eo2-user@ip-172-31-0-224 html)$ ls
404.html  50x.html  icons  nginx-logo.png  poweredby.png
(eo2-user@ip-172-31-0-224 html)$ cat index.html
cat: index.html: No such file or directory
(eo2-user@ip-172-31-0-224 html)$ cat > index.html
-bash: index.html: Permission denied
(eo2-user@ip-172-31-0-224 html)$ sudo cat > index.html
-bash: index.html: Permission denied
(eo2-user@ip-172-31-0-224 html)$ sudo vi index.html
(eo2-user@ip-172-31-0-224 html)$ ls
404.html  50x.html  icons  index.html  nginx-logo.png  poweredby.png
(eo2-user@ip-172-31-0-224 html)$ history
1  yum check-update
2  yum update
3  sudo yum update
4  sudo yum install nginx
5  sudo systemctl nginx
6  sudo systemctl nginx
7  sudo systemctl nginx
8  sudo systemctl status nginx
9  sudo systemctl status nginx
10 sudo systemctl start nginx
11 sudo systemctl status nginx
12 sudo systemctl enable nginx
13 cd /usr/share/nginx/html/
14 ls
15 rm -rf index.html
16 sudo rm -rf index.html
17 ls
18 vim index.html
19 ls
20 cat index.html
21 cat > index.html
22 sudo cat > index.html
23 sudo vi index.html
24 ls
25 history
(eo2-user@ip-172-31-0-224 html)$
```

Step 4: Verify Web Server

- Open a browser and visit <http://your-public-ip> to check if the web server is serving pages.
- You should see the Apache or Nginx default welcome page.

Security & Best Practices

- Restrict SSH access to trusted IPs only.
- Use an Elastic IP for a static public address.
- Implement HTTPS using Let's Encrypt or an SSL certificate.
- Regularly update the server to patch vulnerabilities.

Conclusion

- Successfully deployed a Linux-based web server using Apache/Nginx on AWS EC2.
- Configured security rules for web access.
- Future improvements: Load balancing, auto-scaling, integration with a database.

